Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3512290.3528705

RESEARCH-ARTICLE

Approximating gradients for differentiable quality diversity in reinforcement learning

BRYON TJANAKA, University of Southern California, Los Angeles, CA, United States

MATTHEW C. FONTAINE, University of Southern California, Los Angeles, CA, United States

JULIAN TOGELIUS

STEFANOS NIKOLAIDIS, University of Southern California, Los Angeles, CA, United States

Open Access Support provided by:

University of Southern California

PDF Download
3512290.3528705.pdf
09 March 2026
Total Citations: 32
Total Downloads: 860

Published: 08 July 2022

Citation in BibTeX format

GECCO '22: Genetic and Evolutionary Computation Conference
July 9 - 13, 2022
Massachusetts, Boston

Conference Sponsors:
SIGEVO

GECCO '22: Proceedings of the Genetic and Evolutionary Computation Conference (July 2022)
https://doi.org/10.1145/3512290.3528705
ISBN: 9781450392372

Approximating Gradients for Differentiable Quality Diversity in Reinforcement Learning

Bryon Tjanaka
University of Southern California
Los Angeles, California, USA
tjanaka@usc.edu

Matthew C. Fontaine
University of Southern California
Los Angeles, California, USA
mfontain@usc.edu

Julian Togelius
New York University
Brooklyn, New York, USA
julian@togelius.com

Stefanos Nikolaidis
University of Southern California
Los Angeles, California, USA
nikolaid@usc.edu

ABSTRACT

Consider the problem of training robustly capable agents. One approach is to generate a diverse collection of agent polices. Training can then be viewed as a quality diversity (QD) optimization problem, where we search for a collection of performant policies that are diverse with respect to quantified behavior. Recent work shows that differentiable quality diversity (DQD) algorithms greatly accelerate QD optimization when exact gradients are available. However, agent policies typically assume that the environment is not differentiable. To apply DQD algorithms to training agent policies, we must approximate gradients for performance and behavior. We propose two variants of the current state-of-the-art DQD algorithm that compute gradients via approximation methods common in reinforcement learning (RL). We evaluate our approach on four simulated locomotion tasks. One variant achieves results comparable to the current state-of-the-art in combining QD and RL, while the other performs comparably in two locomotion tasks. These results provide insight into the limitations of current DQD algorithms in domains where gradients must be approximated. Source code is available at https://github.com/icaros-usc/dqd-rl

CCS CONCEPTS

• **Computing methodologies** → **Reinforcement learning**; *Evolutionary robotics.*

KEYWORDS

quality diversity, reinforcement learning, neuroevolution

ACM Reference Format:
Bryon Tjanaka, Matthew C. Fontaine, Julian Togelius, and Stefanos Nikolaidis. 2022. Approximating Gradients for Differentiable Quality Diversity in Reinforcement Learning. In *Genetic and Evolutionary Computation Conference (GECCO '22), July 9–13, 2022, Boston, MA, USA.* ACM, New York, NY, USA, 20 pages. https://doi.org/10.1145/3512290.3528705

This work is licensed under a Creative Commons Attribution International 4.0 License.

GECCO '22, July 9–13, 2022, Boston, MA, USA
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9237-2/22/07.
https://doi.org/10.1145/3512290.3528705

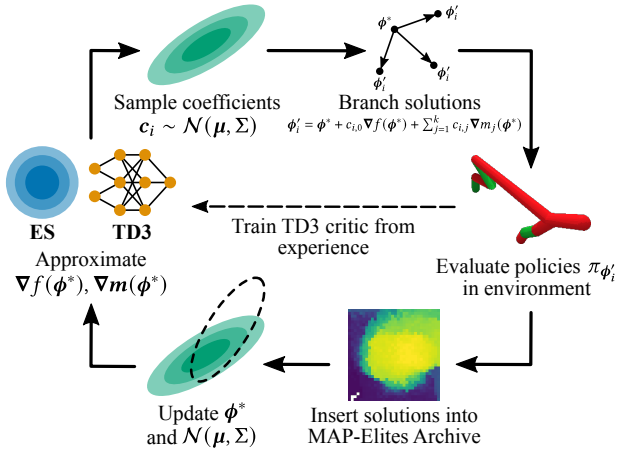

Figure 1: We develop two RL variants of the CMA-MEGA algorithm. Similar to CMA-MEGA, the variants sample gradient coefficients c and branch around a solution point ϕ^*. We evaluate each branched solution ϕ_i' as part of a policy $\pi_{\phi_i'}$ and insert ϕ_i' into the archive. We then update ϕ^* and $\mathcal{N}(\mu, \Sigma)$ to maximize archive improvement. Our RL variants differ from CMA-MEGA by approximating gradients with ES and TD3, since exact gradients are unavailable in RL settings.

1 INTRODUCTION

We focus on the problem of extending differentiable quality diversity (DQD) to reinforcement learning (RL) domains. We propose to approximate gradients for the objective and measure functions, resulting in two variants of the DQD algorithm CMA-MEGA [19].

Consider a half-cheetah agent (Fig. 2) trained for locomotion, where the agent must continue walking forward even when one foot is damaged. If we frame this challenge as an RL problem, two approaches to design a robustly capable agent would be to (1) design a reward function and (2) apply domain randomization [47, 58]. However, prior work [8, 29] suggests that designing such a reward function is difficult, while domain randomization may require manually selecting hundreds of environment parameters [44, 47].

As an alternative approach, consider that we have intuition on what behaviors would be useful for adapting to damage. For instance, we can *measure* how often each foot is used during training,

and we can pre-train a collection of policies that are diverse in how the agent uses its feet. When one of the agent's feet is damaged during deployment, the agent can adapt to the damage by selecting a policy that did not move the damaged foot during training [9, 13].

Pre-training such a collection of policies may be viewed as a quality diversity (QD) optimization problem [9, 13, 40, 49]. Formally, QD assumes an objective function f and one or more measure functions $\boldsymbol{m}$. The goal of QD is to find solutions satisfying all output combinations of $\boldsymbol{m}$, i.e. moving different combinations of feet, while maximizing each solution's f, i.e. walking forward quickly. Most QD algorithms treat f and $\boldsymbol{m}$ as black boxes, but recent work [19] proposes differentiable quality diversity (DQD), which assumes f and $\boldsymbol{m}$ are differentiable functions with exact gradient information. QD algorithms have been applied to procedural content generation [25], robotics [13, 40], aerodynamic shape design [22], and scenario generation in human-robot interaction [17, 18].

The recently proposed DQD algorithm CMA-MEGA [19] outperforms QD algorithms by orders of magnitude when exact gradients are available, such as when searching the latent space of a generative model. However, RL problems like the half-cheetah lack these gradients because the environment is typically non-differentiable, thus limiting the applicability of DQD. To address this limitation, we draw inspiration from how evolution strategies (ES) [1, 39, 51, 60] and deep RL actor-critic methods [21, 38, 53, 54] optimize a reward objective by approximating gradients for gradient descent. *Our key insight is to approximate objective and measure gradients for DQD algorithms by adapting ES and actor-critic methods.*

Our work makes three contributions. **(1)** We formalize the problem of quality diversity for reinforcement learning (QD-RL) and reduce it to an instance of DQD. **(2)** We develop two QD-RL variants of the DQD algorithm CMA-MEGA, where each algorithm approximates objective and measure gradients with a different combination of ES and actor-critic methods. **(3)** We benchmark our variants on four PyBullet locomotion tasks from QDGym [15, 42]. One variant performs comparably (in terms of QD score; Sec. 5.1.3) to the state-of-the-art PGA-MAP-Elites [43] in two tasks. The other variant achieves comparable QD score with PGA-MAP-Elites in all tasks[1] but is less efficient than PGA-MAP-Elites in two tasks.

These results contrast with prior work [19] where CMA-MEGA vastly outperforms a DQD algorithm inspired by PGA-MAP-Elites on benchmark functions where gradient information is available. Overall, we shed light on the limitations of CMA-MEGA in QD domains where the main challenge comes from optimizing the objective rather than from exploring measure space. At the same time, since we decouple gradient estimates from QD optimization, our work opens a path for future research that would benefit from independent improvements to either DQD or RL.

2 PROBLEM STATEMENT

2.1 Quality Diversity (QD)

We adopt the definition of QD from prior work [19]. For a solution vector $\boldsymbol{\phi} \in \mathbb{R}^n$, QD considers an objective function $f(\boldsymbol{\phi})$ and k measures[2] $m_i(\boldsymbol{\phi}) \in \mathbb{R}$ (for $i \in 1..k$) or, as a joint measure, $\boldsymbol{m}(\boldsymbol{\phi}) \in \mathbb{R}^k$. These measures form a k-dimensional measure space $\mathcal{X}$. For every $\boldsymbol{x} \in \mathcal{X}$, the QD objective is to find solution $\boldsymbol{\phi}$ such that $\boldsymbol{m}(\boldsymbol{\phi}) = \boldsymbol{x}$ and $f(\boldsymbol{\phi})$ is maximized. Since $\mathcal{X}$ is continuous, it would require infinite memory to solve the QD problem, so algorithms in the MAP-Elites family [13, 40] discretize $\mathcal{X}$ by forming a tesselation $\mathcal{Y}$ consisting of M cells. Thus, we relax the QD problem to one of searching for an *archive* $\mathcal{A}$ consisting of M *elites* $\boldsymbol{\phi}_i$, one for each cell in $\mathcal{Y}$. Then, the QD objective is to maximize the performance $f(\boldsymbol{\phi}_i)$ of all elites:

$$\max_{\boldsymbol{\phi}_{1..M}} \sum_{i=1}^{M} f(\boldsymbol{\phi}_i) \tag{1}$$

2.1.1 Differentiable Quality Diversity (DQD). In DQD, we assume f and $\boldsymbol{m}$ are first-order differentiable. We denote the objective gradient as $\boldsymbol{\nabla} f(\boldsymbol{\phi})$, or abbreviated as $\boldsymbol{\nabla} f$, and the measure gradients as $\boldsymbol{\nabla}\boldsymbol{m}(\boldsymbol{\phi})$ or $\boldsymbol{\nabla}\boldsymbol{m}$.

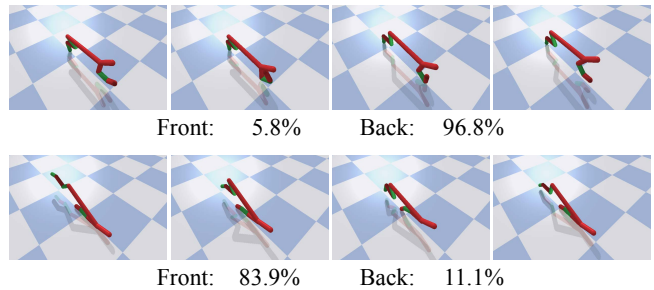

Figure 2: A half-cheetah agent executing two walking policies. In the top row, the agent walks on its back foot while tapping the ground with its front foot. In the bottom row, the agent walks on its front foot while jerking its back foot. Values below each row show the percentage of time each foot contacts the ground (each foot is measured individually, so values do not sum to 100%). With these policies, the agent could continue walking even if one foot is damaged.

2.2 Quality Diversity for Reinforcement Learning (QD-RL)

We define QD-RL as an instance of the QD problem in which each solution $\boldsymbol{\phi}$ parameterizes an RL policy $\pi_{\boldsymbol{\phi}}$. Then, the objective $f(\boldsymbol{\phi})$ is the *expected discounted return* of $\pi_{\boldsymbol{\phi}}$, and the measures $\boldsymbol{m}(\boldsymbol{\phi})$ are functions of $\pi_{\boldsymbol{\phi}}$. Formally, drawing on the Markov Decision Process (MDP) formulation [55], we represent QD-RL as a tuple $(\mathcal{S}, \mathcal{U}, p, r, \gamma, \boldsymbol{m})$. On discrete timesteps t in an episode of interaction, an agent observes state $s \in \mathcal{S}$ and takes action $a \in \mathcal{U}$ according to a policy $\pi_{\boldsymbol{\phi}}(a|s)$. The agent then receives scalar reward $r(s, a)$ and observes next state $s' \in \mathcal{S}$ according to $s' \sim p(\cdot|s, a)$. Each episode thus has a trajectory $\xi = \{s_0, a_0, s_1, a_1, .., s_T\}$, where T is the number of timesteps in the episode, and the probability that policy $\pi_{\boldsymbol{\phi}}$ takes trajectory ξ is $p_{\boldsymbol{\phi}}(\xi) = p(s_0) \prod_{t=0}^{T-1} \pi_{\boldsymbol{\phi}}(a_t|s_t) p(s_{t+1}|s_t, a_t)$.

[1] We note that the performance of the CMA-MEGA is worse than PGA-MAP-Elites in two of the tasks, albeit within variance. We consider it likely that additional runs would result in PGA-MAP-Elites performing significantly better in these tasks. We leave further evaluation for future work.

[2] Prior work refers to measure function outputs as "behavior characteristics," "behavior descriptors," or "feature descriptors."

Now, we define the *expected discounted return* of policy $\pi_{\boldsymbol{\phi}}$ as

$$f(\boldsymbol{\phi}) = \mathbb{E}_{\xi \sim p_{\boldsymbol{\phi}}} \left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t) \right] \tag{2}$$

where the discount factor $\gamma \in (0, 1)$ trades off between short- and long-term rewards. Finally, we quantify the behavior of policy $\pi_{\boldsymbol{\phi}}$ via a k-dimensional measure function $\boldsymbol{m}(\boldsymbol{\phi})$.

2.2.1 QD-RL as an instance of DQD. We reduce QD-RL to a DQD problem. Since the exact gradients $\boldsymbol{\nabla} f$ and $\boldsymbol{\nabla m}$ usually do not exist in QD-RL, we must instead approximate them.

3 BACKGROUND

3.1 Single-Objective Reinforcement Learning

We review algorithms which train a policy to maximize a single objective, i.e. $f(\boldsymbol{\phi})$ in Eq. 2, with the goal of applying these algorithms' gradient approximations to DQD in Sec. 4.

3.1.1 Evolution strategies (ES). ES [4] is a class of evolutionary algorithms which optimizes the objective by sampling a population of solutions and moving the population towards areas of higher performance. Natural Evolution Strategies (NES) [60, 61] is a type of ES which updates the sampling distribution of solutions by taking steps on distribution parameters in the direction of the natural gradient [2]. For example, with a Gaussian sampling distribution, each iteration of an NES would compute natural gradients to update the mean $\boldsymbol{\mu}$ and covariance Σ.

We consider an NES-inspired algorithm [51] which has demonstrated success in RL domains. This algorithm, which we refer to as OpenAI-ES, samples λ_{es} solutions from an isotropic Gaussian but only computes a gradient step for the mean $\boldsymbol{\phi}$. Each solution sampled by OpenAI-ES is represented as $\boldsymbol{\phi} + \sigma \boldsymbol{\epsilon}_i$, where σ is the fixed standard deviation of the Gaussian and $\boldsymbol{\epsilon}_i \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$. Once these solutions are evaluated, OpenAI-ES estimates the gradient as

$$\boldsymbol{\nabla} f(\boldsymbol{\phi}) \approx \frac{1}{\lambda_{es}\sigma} \sum_{i=1}^{\lambda_{es}} f(\boldsymbol{\phi} + \sigma \boldsymbol{\epsilon}_i) \boldsymbol{\epsilon}_i \tag{3}$$

OpenAI-ES then passes this estimate to an Adam optimizer [32] which outputs a gradient ascent step for $\boldsymbol{\phi}$. To make the estimate more accurate, OpenAI-ES further includes techniques such as mirror sampling and rank normalization [5, 26, 60].

3.1.2 Actor-critic methods. While ES treats the objective as a black box, actor-critic methods leverage the MDP structure of the objective, i.e. the fact that $f(\boldsymbol{\phi})$ is a sum of Markovian values. We are most interested in Twin Delayed Deep Deterministic policy gradient (TD3) [21], an off-policy actor-critic method. TD3 maintains (1) an actor consisting of the policy $\pi_{\boldsymbol{\phi}}$ and (2) a critic consisting of state-action value functions $Q_{\boldsymbol{\theta}_1}(s, a)$ and $Q_{\boldsymbol{\theta}_2}(s, a)$ which differ only in random initialization. Through interactions in the environment, the actor generates experience which is stored in a replay buffer $\mathcal{B}$. This experience is sampled to train $Q_{\boldsymbol{\theta}_1}$ and $Q_{\boldsymbol{\theta}_2}$. Simultaneously, the actor improves by maximizing $Q_{\boldsymbol{\theta}_1}$ via gradient ascent ($Q_{\boldsymbol{\theta}_2}$ is only used during critic training). Specifically, for an objective f' which is based on the critic and approximates f, TD3 estimates a gradient $\boldsymbol{\nabla} f'(\boldsymbol{\phi})$ and passes it to an Adam optimizer. Notably, TD3 never updates network weights directly, instead accumulating weights into *target networks* $\pi_{\boldsymbol{\phi}'}, Q_{\boldsymbol{\theta}'_1}, Q_{\boldsymbol{\theta}'_2}$ via an exponentially weighted moving average with update rate τ.

3.2 Quality Diversity Algorithms

3.2.1 MAP-Elites extensions for QD-RL. One of the simplest QD algorithms is MAP-Elites [13, 40]. MAP-Elites creates an archive $\mathcal{A}$ by tesselating the measure space $\mathcal{X}$ into a grid of evenly-sized cells. Then, it draws λ initial solutions from a multivariate Gaussian $\mathcal{N}(\boldsymbol{\phi}_0, \sigma \boldsymbol{I})$ centered at some $\boldsymbol{\phi}_0$. Next, for each sampled solution $\boldsymbol{\phi}$, MAP-Elites computes $f(\boldsymbol{\phi})$ and $\boldsymbol{m}(\boldsymbol{\phi})$ and inserts $\boldsymbol{\phi}$ into $\mathcal{A}$. In subsequent iterations, MAP-Elites randomly selects λ solutions from $\mathcal{A}$ and adds Gaussian noise, i.e. solution $\boldsymbol{\phi}$ becomes $\boldsymbol{\phi} + \mathcal{N}(\mathbf{0}, \sigma \boldsymbol{I})$. Solutions are placed into cells based on their measures; if a solution has higher f than the solution currently in the cell, it replaces that solution. Once inserted into $\mathcal{A}$, solutions are known as *elites*.

Due to the high dimensionality of neural network parameters, direct policy optimization with MAP-Elites has not proven effective in QD-RL [9], although indirect encodings have been shown to scale to large policy networks [23, 50]. For direct search, several extensions merge MAP-Elites with actor-critic methods and ES. For instance, Policy Gradient Assisted MAP-Elites (PGA-MAP-Elites) [43] combines MAP-Elites with TD3. Each iteration, PGA-MAP-Elites evaluates λ solutions for insertion into the archive. $\frac{\lambda}{2}$ of these are created by selecting random solutions from the archive and taking gradient ascent steps with a TD3 critic. The other $\frac{\lambda}{2}$ solutions are created with a directional variation operator [59] which selects two solutions $\boldsymbol{\phi}_1$ and $\boldsymbol{\phi}_2$ from the archive and creates a new one according to $\boldsymbol{\phi}' = \boldsymbol{\phi}_1 + \sigma_1 \mathcal{N}(\mathbf{0}, \boldsymbol{I}) + \sigma_2(\boldsymbol{\phi}_2 - \boldsymbol{\phi}_1)\mathcal{N}(0, 1)$. Finally, PGA-MAP-Elites maintains a "greedy actor" which provides actions when training the critics (identically to the actor in TD3). Every iteration, PGA-MAP-Elites inserts this greedy actor into the archive. PGA-MAP-Elites achieves state-of-the-art performance on locomotion tasks in the QDGym benchmark [42].

Another MAP-Elites extension is ME-ES [9], which combines MAP-Elites with an OpenAI-ES optimizer. In the "explore-exploit" variant, ME-ES alternates between two phases. In the "exploit" phase, ME-ES restarts OpenAI-ES at a mean $\boldsymbol{\phi}$ and optimizes the objective for k iterations, inserting the current $\boldsymbol{\phi}$ into the archive in each iteration. In the "explore" phase, ME-ES repeats this process, but OpenAI-ES instead optimizes for novelty, where novelty is the distance in measure space from a new solution to previously encountered solutions. ME-ES also has an "exploit" variant and an "explore" variant, which each execute only one type of phase.

Our work is related to ME-ES in that we also adapt OpenAI-ES, but instead of alternating between following a novelty gradient and objective gradient, we compute all objective and measure gradients and allow a CMA-ES [28] instance to decide which gradients to follow by sampling gradient coefficients from a multivariate Gaussian updated over time (Sec. 3.2.2). We include MAP-Elites, PGA-MAP-Elites, and ME-ES as baselines in our experiments. Refer to Fig. 3 for a diagram which compares these algorithms to our approach.

3.2.2 Covariance Matrix Adaptation MAP-Elites via a Gradient Arborescence (CMA-MEGA). We directly extend CMA-MEGA [19] to address QD-RL. CMA-MEGA is a DQD algorithm based on the QD algorithm CMA-ME [20]. The intuition behind CMA-MEGA is that if we knew which direction the current solution point $\boldsymbol{\phi}^*$

should move in objective-measure space, then we could calculate that change in search space via a linear combination of objective and measure gradients. From CMA-ME, we know a good direction is one that results in the largest archive improvement.

Each iteration, CMA-MEGA first calculates objective and measure gradients for a solution point $\boldsymbol{\phi}^*$. Next, it generates λ new solutions by sampling gradient coefficients $\boldsymbol{c} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ and computing $\boldsymbol{\phi}' \leftarrow \boldsymbol{\phi}^* + c_0 \nabla f(\boldsymbol{\phi}^*) + \sum_{j=1}^{k} c_j \nabla m_j(\boldsymbol{\phi}^*)$. CMA-MEGA inserts these solutions into the archive and computes their *improvement,* Δ. Δ is defined as $f(\boldsymbol{\phi}')$ if $\boldsymbol{\phi}'$ populates a new cell, and $f(\boldsymbol{\phi}') - f(\boldsymbol{\phi}'_{\mathcal{E}})$ if $\boldsymbol{\phi}'$ improves an existing cell (replaces a previous solution $\boldsymbol{\phi}'_{\mathcal{E}}$). After CMA-MEGA inserts the solutions, it ranks them by Δ. If a solution populates a new cell, its Δ always ranks higher than that of a solution which only improves an existing cell. CMA-MEGA then moves the solution point $\boldsymbol{\phi}^*$ towards the largest archive improvement, but also adapts the distribution $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ towards better gradient coefficients by the same ranking. By leveraging gradient information, CMA-MEGA solves QD benchmarks with orders of magnitude fewer solution evaluations than previous QD algorithms.

3.2.3 Beyond MAP-Elites. Several QD-RL algorithms have been developed outside the MAP-Elites family. NS-ES [11] builds on Novelty Search (NS) [35, 36], a family of QD algorithms which add solutions to an *unstructured archive* only if they are far away from existing archive solutions in measure space. Using OpenAI-ES, NS-ES concurrently optimizes several agents for novelty. Its variants NSR-ES and NSRA-ES optimize for a linear combination of novelty and objective. Meanwhile, the QD-RL algorithm [7] (distinct from the QD-RL problem we define) maintains an archive with all past solutions and optimizes agents along a Pareto front of the objective and novelty. Finally, Diversity via Determinants (DvD) [46] leverages a kernel method to maintain diversity in a population of solutions. As NS-ES, QD-RL, and DvD do not output a MAP-Elites grid archive, we leave their investigation for future work.

3.3 Diversity in Reinforcement Learning

Here we distinguish QD-RL from prior work which also applies diversity to RL. One area of work is in latent- and goal-conditioned policies. For latent-conditioned policy $\pi_{\boldsymbol{\phi}}(a|s, z)$ [16, 33, 37] or goal-conditioned policy $\pi_{\boldsymbol{\phi}}(a|s, g)$ [3, 52], varying the latent variable z or goal g results in different behaviors, e.g. different walking gaits or walking to a different location. While QD-RL also seeks a range of behaviors, the measures $\boldsymbol{m}(\boldsymbol{\phi})$ are computed *after* evaluating $\boldsymbol{\phi}$, rather than *before* the evaluation. In general, QD-RL focuses on finding a variety of policies for a single task, rather than attempting to solve a variety of tasks with a single conditioned policy.

Another area of work combines evolutionary and actor-critic algorithms to solve single-objective hard-exploration problems [10, 30, 31, 48, 56]. In these methods, an evolutionary algorithm such as cross-entropy method [14] facilitates exploration by generating a diverse population of policies, while an actor-critic algorithm such as TD3 trains high-performing policies with this population's environment experience. QD-RL differs from these methods in that it views diversity as a component of the output, while these methods view diversity as a means for environment exploration. Hence, QD-RL measures policy behavior via a measure function and collects diverse policies in an archive. In contrast, these RL exploration methods assume that trajectory diversity, rather than targeting specific behavioral diversity, is enough to drive exploration to discover a single optimal policy.

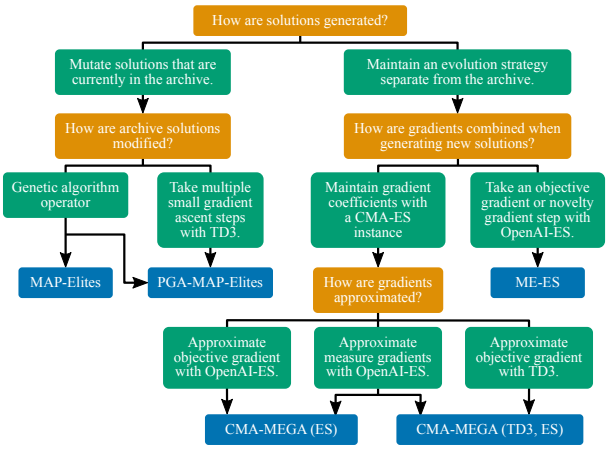

Figure 3: Diagram of MAP-Elites extensions for QD-RL, showing how our CMA-MEGA variants differ from other QD-RL algorithms.

4 APPROXIMATING GRADIENTS FOR DQD

Since DQD algorithms require exact objective and measure gradients, we cannot directly apply CMA-MEGA to QD-RL. To address this limitation, we replace exact gradients with gradient approximations (Sec. 4.1) and develop two CMA-MEGA variants (Sec. 4.2).

4.1 Approximating Objective and Measure Gradients

We adapt gradient approximations from ES and actor-critic methods. Since the objective has an MDP structure, we estimate objective gradients ∇f with ES and actor-critic methods. Since the measures are black boxes, we estimate measure gradients $\nabla \boldsymbol{m}$ with ES.

4.1.1 Approximating objective gradients with ES and actor-critic methods. We estimate objective gradients with two methods. First, we treat the objective as a black box and estimate its gradient with a black box method, namely the OpenAI-ES gradient estimate in Eq. 3. Since OpenAI-ES performs well in RL domains [34, 45, 51], we believe this estimate is suitable for approximating gradients for CMA-MEGA in QD-RL settings. Importantly, this estimate requires environment interaction by evaluating λ_{es} solutions.

Since the objective has a well-defined structure, i.e. it is a sum of rewards from an MDP (Eq. 2), we also estimate its gradient with an actor-critic method, TD3. TD3 is well-suited for this purpose because it efficiently estimates objective gradients for the multiple policies that CMA-MEGA and other QD-RL algorithms generate. In particular, once the critic is trained, TD3 can provide a gradient estimate for any policy without additional environment interaction.

Among actor-critic methods, we select TD3 since it achieves high performance while optimizing primarily for the RL objective. Prior work [21] shows that TD3 outperforms on-policy actor-critic methods [53, 54]. While the off-policy Soft Actor-Critic [27] algorithm

can outperform TD3, it optimizes a maximum-entropy objective designed to encourage exploration. In our work, we explore by finding policies with different measures. Thus, we leave for future work the problem of integrating QD-RL with the action diversity encouraged by entropy maximization.

4.1.2 Approximating measure gradients with ES. Since measures do not have special properties such as an MDP structure (Sec. 2.2), we only estimate their gradient with black box methods. Thus, similar to the objective, we approximate each measure's gradient ∇m_i with the OpenAI-ES gradient estimate, replacing f with m_i in Eq. 3.

Since the OpenAI-ES gradient estimate requires additional environment interaction, all of our CMA-MEGA variants require environment interaction to estimate gradients. However, the environment interaction required to estimate measure gradients remains constant even as the number of measures increases, since we can reuse the same λ_{es} solutions to estimate each ∇m_i.

In problems where the measures have an MDP structure similar to the objective, it may be feasible to estimate each ∇m_i with its own TD3 instance. In the environments in our work (Sec. 5.1), each measure is non-Markovian since it calculates the proportion of time a walking agent's foot spends on the ground. This calculation depends on the entire agent trajectory rather than on one state.

4.2 CMA-MEGA Variants

Our choice of gradient approximations leads to two CMA-MEGA variants. **CMA-MEGA (ES)** approximates objective and measure gradients with OpenAI-ES, while **CMA-MEGA (TD3, ES)** approximates the objective gradient with TD3 and measure gradients with OpenAI-ES. Fig. 1 shows an overview of both algorithms, and Algorithm 1 shows their pseudocode. As CMA-MEGA (TD3, ES) builds on CMA-MEGA (ES), we present only CMA-MEGA (TD3, ES) and highlight lines that CMA-MEGA (TD3, ES) additionally executes.

Identically to CMA-MEGA, the two variants maintain three primary components: a solution point $\boldsymbol{\phi}^*$, a multivariate Gaussian distribution $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ for sampling gradient coefficients, and a MAP-Elites archive $\mathcal{A}$ for storing solutions. We initialize the archive and solution point on line 3, and we initialize the coefficient distribution as part of a CMA-ES instance on line 4.[3]

In the main loop (line 6), we follow the workflow shown in Fig. 1. First, after evaluating $\boldsymbol{\phi}^*$ and inserting it into the archive (line 7-8), we approximate its gradients with either ES or TD3 (line 9-10). *This gradient approximation forms the key difference between our variants and the original CMA-MEGA algorithm [19].*

Next, we branch from $\boldsymbol{\phi}^*$ to create solutions $\boldsymbol{\phi}'_i$ by sampling $\boldsymbol{c}_i$ from the coefficient distribution and computing perturbations ∇_i (line 13-15). We then evaluate each $\boldsymbol{\phi}'_i$ and insert it into the archive (line 16-17).

Finally, we update the solution point and the coefficient distribution's CMA-ES instance by forming an *improvement ranking* based on the improvement Δ_i (Sec. 3.2.2; line 19-21). Importantly, since we rank based on improvement, this update enables the CMA-MEGA variants to maximize the QD objective (Eq. 1) [19].

[3]We set the CMA-ES batch size λ' slightly lower than the total batch size λ (line 2). While CMA-MEGA (ES) and CMA-MEGA (TD3, ES) both evaluate λ solutions each iteration, one evaluation is reserved for $\boldsymbol{\phi}^*$ (line 7). In CMA-MEGA (TD3, ES), one more evaluation is reserved for the greedy actor (line 26).

Algorithm 1: CMA-MEGA (ES) and CMA-MEGA (TD3, ES). Highlighted portions are only executed in CMA-MEGA (TD3, ES). Adapted from CMA-MEGA [19]. Refer to Appendix A for functions whose names are in Small_Caps.

1 **CMA-MEGA variants** $(evaluate, \boldsymbol{\phi}_0, N, \lambda, \sigma_g, \eta, \lambda_{es}, \sigma_e)$**:**
  Input: Function *evaluate* which executes a policy $\boldsymbol{\phi}$ and outputs objective $f(\boldsymbol{\phi})$ and measures $\boldsymbol{m}(\boldsymbol{\phi})$, initial solution $\boldsymbol{\phi}_0$, desired iterations N, batch size λ, initial CMA-ES step size σ_g, learning rate η, ES batch size λ_{es}, ES standard deviation σ_e
  Result: Generates $N\lambda$ solutions, storing elites in an archive $\mathcal{A}$
2 $\lambda' \leftarrow \lambda - 1$ $- 1$
3 Initialize empty archive $\mathcal{A}$, solution point $\boldsymbol{\phi}^* \leftarrow \boldsymbol{\phi}_0$
4 Initialize CMA-ES with population λ', resulting in $\boldsymbol{\mu} = \boldsymbol{0}, \Sigma = \sigma_g \boldsymbol{I}$, and internal CMA-ES parameters $\boldsymbol{p}$
5 $\mathcal{B}, Q_{\theta_1}, Q_{\theta_2}, \pi_{\boldsymbol{\phi}_q}, Q_{\theta'_1}, Q_{\theta'_2}, \pi_{\boldsymbol{\phi}'_q} \leftarrow$ Initialize_TD3()
6 **for** $iter \leftarrow 1..N$ **do**
7 $f(\boldsymbol{\phi}^*), \boldsymbol{m}(\boldsymbol{\phi}^*) \leftarrow evaluate(\boldsymbol{\phi}^*)$
8 Update_Archive$(\mathcal{A}, \boldsymbol{\phi}^*, f(\boldsymbol{\phi}^*), \boldsymbol{m}(\boldsymbol{\phi}^*))$
9 $\nabla f(\boldsymbol{\phi}^*), \nabla \boldsymbol{m}(\boldsymbol{\phi}^*) \leftarrow$ ES_Gradients$(\boldsymbol{\phi}^*, \lambda_{es}, \sigma_e)$
10 $\nabla f(\boldsymbol{\phi}^*) \leftarrow$ TD3_Gradient$(\boldsymbol{\phi}^*, Q_{\theta_1}, \mathcal{B})$
11 Normalize $\nabla f(\boldsymbol{\phi}^*)$ and $\nabla \boldsymbol{m}(\boldsymbol{\phi}^*)$ to be unit vectors
12 **for** $i \leftarrow 1..\lambda'$ **do**
13 $\boldsymbol{c}_i \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$
14 $\nabla_i \leftarrow c_{i,0} \nabla f(\boldsymbol{\phi}^*) + \sum_{j=1}^{k} c_{i,j} \nabla m_j(\boldsymbol{\phi}^*)$
15 $\boldsymbol{\phi}'_i \leftarrow \boldsymbol{\phi}^* + \nabla_i$
16 $f(\boldsymbol{\phi}'_i), \boldsymbol{m}'(\boldsymbol{\phi}'_i) \leftarrow evaluate(\boldsymbol{\phi}'_i)$
17 $\Delta_i \leftarrow$ Update_Archive$(\mathcal{A}, \boldsymbol{\phi}'_i, f(\boldsymbol{\phi}'_i), \boldsymbol{m}(\boldsymbol{\phi}'_i))$
18 **end**
19 Rank $\boldsymbol{c}_i, \nabla_i$ by Δ_i
20 Adapt CMA-ES parameters $\boldsymbol{\mu}, \Sigma, \boldsymbol{p}$ based on rankings of $\boldsymbol{c}_i$
21 $\boldsymbol{\phi}^* \leftarrow \boldsymbol{\phi}^* + \eta \sum_{i=1}^{\lambda} w_i \nabla_{\text{rank[i]}}$ `// $w_i$ is part of $\boldsymbol{p}$`
22 **if** *there is no change in* $\mathcal{A}$ **then**
23 Restart CMA-ES with $\boldsymbol{\mu} = \boldsymbol{0}, \Sigma = \sigma_g \boldsymbol{I}$
24 Set $\boldsymbol{\phi}^*$ to a randomly selected elite from $\mathcal{A}$
25 **end**
26 $f(\boldsymbol{\phi}_q), \boldsymbol{m}(\boldsymbol{\phi}_q) \leftarrow evaluate(\boldsymbol{\phi}_q)$
27 Update_Archive$(\mathcal{A}, \boldsymbol{\phi}_q, f(\boldsymbol{\phi}_q), \boldsymbol{m}(\boldsymbol{\phi}_q))$
28 Add experience from all calls to *evaluate* into $\mathcal{B}$
29 Train_TD3$(Q_{\theta_1}, Q_{\theta_2}, \pi_{\boldsymbol{\phi}_q}, Q_{\theta'_1}, Q_{\theta'_2}, \pi_{\boldsymbol{\phi}'_q}, \mathcal{B})$
30 **end**

Our CMA-MEGA variants have two additional components. First, we check if no solutions were inserted into the archive at the end of the iteration, which would indicate that we should reset the coefficient distribution and the solution point (line 22-24). Second, in the case of CMA-MEGA (TD3, ES), we manage a TD3 instance similar to how PGA-MAP-Elites does (Sec. 3.2.1). This TD3 instance consists of a replay buffer $\mathcal{B}$, critic networks Q_{θ_1} and Q_{θ_2}, a greedy actor $\pi_{\boldsymbol{\phi}_q}$, and target networks $Q_{\theta'_1}$, $Q_{\theta'_2}$, $\pi_{\boldsymbol{\phi}'_q}$ (all initialized on line

5). At the end of each iteration, we use the greedy actor to train the critics, and we also insert it into the archive (line 26-29).

5 EXPERIMENTS

We compare our two proposed CMA-MEGA variants (CMA-MEGA (ES), CMA-MEGA (TD3, ES)) with three baselines (PGA-MAP-Elites, ME-ES, MAP-Elites) in four locomotion tasks. We implement MAP-Elites as described in Sec. 3.2.1, and we select the explore-exploit variant for ME-ES since it has performed at least as well as both the explore variant and the exploit variant in several domains [9].

5.1 Evaluation Domains

5.1.1 QDGym. We evaluate our algorithms in four locomotion environments from QDGym [42], a library built on PyBullet Gym [12, 15] and OpenAI Gym [6]. Appendix C lists all environment details. In each environment, the QD algorithm outputs an archive of walking policies for a simulated agent. The agent is primarily rewarded for its forward speed. There are also reward shaping [41] signals, such as a punishment for applying higher joint torques, intended to guide policy optimization. The measures compute the proportion of time (number of timesteps divided by total timesteps in an episode) that each of the agent's feet contacts the ground.

QDGym is challenging because the objective in each environment does not "align" with the measures, in that finding policies with different measures (i.e. exploring the archive) does not necessarily lead to optimization of the objective. While it may be trivial to fill the archive with low-performing policies which stand in place and lift the feet up and down to achieve different measures, the agents' complexity (high degrees of freedom) makes it difficult to learn a high-performing policy for each value of the measures.

5.1.2 Hyperparameters. Each agent's policy is a neural network which takes in states and outputs actions. There are two hidden layers of 128 nodes, and the hidden and output layers have `tanh` activation. We initialize weights with Xavier initialization [24].

For the archive, we tesselate each environment's measure space into a grid of evenly-sized cells (see Table 6 for grid dimensions). Each measure is bound to the range $[0, 1]$, the min and max proportion of time that one foot can contact the ground.

Each algorithm evaluates 1 million solutions in the environment. Due to computational limits, we evaluate each solution once instead of averaging multiple episodes, so each algorithm runs 1 million episodes total. Refer to Appendix B for further hyperparameters.

5.1.3 Metrics. Our primary metric is *QD score* [49], which provides a holistic view of algorithm performance. QD score is the sum of the objective values of all elites in the archive, i.e. $\sum_{i=1}^{M} \mathbf{1}_{\boldsymbol{\phi}_i \text{exists}} f(\boldsymbol{\phi}_i)$, where M is the number of archive cells. We note that the contribution of a cell to the QD score is 0 if the cell is unoccupied. We set the objective f to be the *expected undiscounted return*, i.e. we set $\gamma = 1$ in Eq. 2.

Since objectives may be negative, an algorithm's QD score may be penalized when adding a new solution. To prevent this, we define a *minimum objective* in each environment by taking the lowest objective value that was inserted into the archive in any experiment in that environment. We subtract this minimum from every solution, such that every solution that was inserted into an archive has an objective value of at least 0. Thus, we use QD score defined as $\sum_{i=1}^{M} \mathbf{1}_{\boldsymbol{\phi}_i \text{exists}} (f(\boldsymbol{\phi}_i) - \text{min objective})$. We also define a *maximum objective* equivalent to each environment's "reward threshold" in PyBullet Gym. This threshold is the objective value at which an agent is considered to have successfully learned to walk.

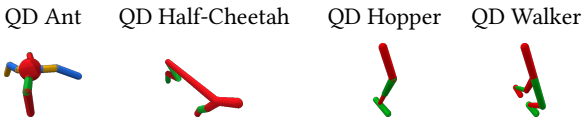

Figure 4: QDGym locomotion environments [42].

We report two metrics in addition to QD score. *Archive coverage*, the proportion of cells for which the algorithm found an elite, gauges how well the QD algorithm explores measure space, and *best performance*, the highest objective of any elite in the archive, gauges how well the QD algorithm exploits the objective.

5.2 Experimental Design

We follow a between-groups design, where the two independent variables are environment (QD Ant, QD Half-Cheetah, QD Hopper, QD Walker) and algorithm (CMA-MEGA (ES), CMA-MEGA (TD3, ES), PGA-MAP-Elites, ME-ES, MAP-Elites). The dependent variable is the QD score. In each environment, we run each algorithm for 5 trials with different random seeds and test three hypotheses:

H1: CMA-MEGA (ES) will outperform all baselines (PGA-MAP-Elites, ME-ES, MAP-Elites).

H2: CMA-MEGA (TD3, ES) will outperform all baselines.

H3: CMA-MEGA (TD3, ES) will outperform CMA-MEGA (ES).

H1 and H2 are based on prior work [19] which showed that in QD benchmark domains, CMA-MEGA outperforms algorithms that do not leverage both objective and measure gradients. H3 is based on results [45] which suggest that actor-critic methods outperform ES in PyBullet Gym. Thus, we expect the TD3 objective gradient to be more accurate than the ES objective gradient, leading to more efficient traversal of objective-measure space and higher QD score.

5.3 Implementation

We implement all QD algorithms with the pyribs library [57] except for ME-ES, which we adapt from the authors' implementation. We run each experiment with 100 CPUs on a high-performance cluster. We allocate one NVIDIA Tesla P100 GPU to algorithms that train TD3 (CMA-MEGA (TD3, ES) and PGA-MAP-Elites). Depending on the algorithm and environment, each experiment lasts 4-20 hours; refer to Table 12, Appendix E for mean runtimes. We have released our source code at https://github.com/icaros-usc/dqd-rl

6 RESULTS

We ran 5 trials of each algorithm in each environment. In each trial, we allocated 1 million evaluations and recorded the QD score, archive coverage, and best performance. Fig. 5 plots these metrics, and Appendix E lists final values of all metrics. Appendix G shows example heatmaps and histograms of each archive, and the supplemental material contains videos of generated agents.

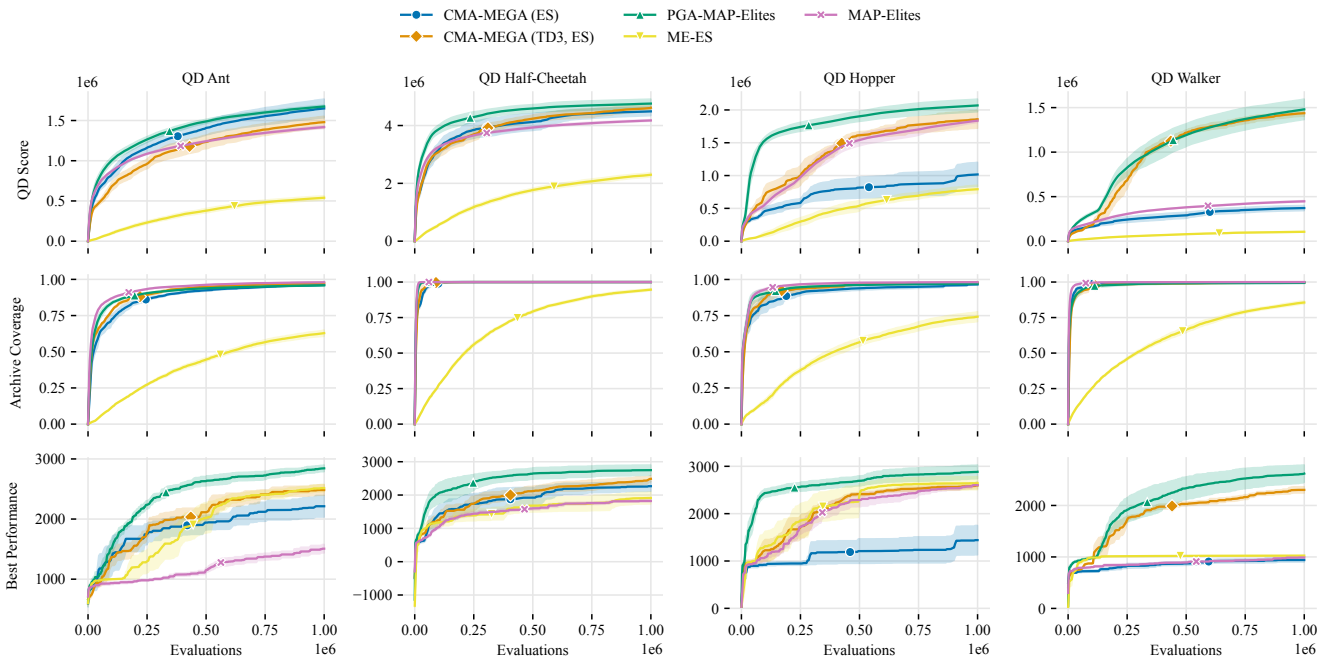

Figure 5: Plots of QD score, archive coverage, and best performance for the 5 algorithms in our experiments in all 4 environments from QDGym. The x-axis in all plots is the number of solutions evaluated. Solid lines show the mean over 5 trials, and shaded regions show the standard error of the mean.

6.1 Analysis

To test our hypotheses, we conducted a two-way ANOVA which examined the effect of algorithm and environment on the QD score. We note that the ANOVA requires QD scores to have the same scale, but each environment's QD score has a different scale by default. Thus, for this analysis, we normalized QD scores by dividing by each environment's maximum QD score, defined as *grid cells* * (*max objective - min objective*) (see Appendix C for these quantities).

We found a statistically significant interaction between algorithm and environment on QD score, $F(12, 80) = 16.82, p < 0.001$. Simple main effects analysis indicated that the algorithm had a significant effect on QD score in each environment, so we ran pairwise comparisons (two-sided t-tests) with Bonferroni corrections (Appendix F). Our results are as follows:

H1: There is no significant difference in QD score between CMA-MEGA (ES) and PGA-MAP-Elites in QD Ant and QD Half-Cheetah, but in QD Hopper and QD Walker, CMA-MEGA (ES) attains significantly lower QD score than PGA-MAP-Elites. CMA-MEGA (ES) achieves significantly higher QD score than ME-ES in all environments except QD Hopper, where there is no significant difference. There is no significant difference between CMA-MEGA (ES) and MAP-Elites in all domains except QD Hopper, where CMA-MEGA (ES) attains significantly lower QD score.

H2: In all environments, there is no significant difference in QD score between CMA-MEGA (TD3, ES) and PGA-MAP-Elites. CMA-MEGA (TD3, ES) achieves significantly higher QD score than ME-ES in all environments. CMA-MEGA (TD3, ES) achieves significantly higher QD score than MAP-Elites in QD Half-Cheetah and Walker, with no significant difference in QD Ant and QD Hopper.

H3: CMA-MEGA (TD3, ES) achieves significantly higher QD score than CMA-MEGA (ES) in QD Hopper and QD Walker, but there is no significant difference in QD Ant and QD Half-Cheetah.

6.2 Discussion

We discuss how the CMA-MEGA variants differ from the baselines (Sec. 6.2.1-6.2.4) and how they differ from each other (Sec. 6.2.5).

6.2.1 PGA-MAP-Elites and objective-measure space exploration. Of the CMA-MEGA variants, CMA-MEGA (TD3, ES) performed the closest to PGA-MAP-Elites, with no significant QD score difference in any environment. This result differs from prior work [19] in QD benchmark domains, where CMA-MEGA outperformed OG-MAP-Elites, a baseline DQD algorithm inspired by PGA-MAP-Elites.

We attribute this difference to the difficulty of exploring objective-measure space in the benchmark domains. For example, the linear projection benchmark domain is designed to be "distorted" [20]. Values in the center of its measure space are easy to obtain with random sampling, while values at the edges are unlikely to be sampled. Hence, high QD score arises from exploring measure space and filling the archive. Since CMA-MEGA adapts its sampling distribution, it is able to perform this exploration, while OG-MAP-Elites remains "stuck" in the center of the measure space.

In contrast, as discussed in Sec. 5.1.1, it is relatively easy to fill the archive in QDGym. We see this empirically: in all environments,

all algorithms achieve nearly 100% archive coverage, usually within the first 250k evaluations (Fig. 5). Hence, the best QD score is achieved by increasing the objective value of solutions after filling the archive. PGA-MAP-Elites achieves this by optimizing half of its generated solutions with respect to its TD3 critic. The genetic operator likely further enhances the efficacy of this optimization, by taking previously-optimized solutions and combining them to obtain high-performing solutions in other parts of the archive.

On the other hand, the CMA-MEGA variants place less emphasis on maximizing the performance of each solution, compared to PGA-MAP-Elites: in each trial, PGA-MAP-Elites takes 5 million objective gradient steps with respect to its TD3 critic, while the CMA-MEGA variants only compute 5k objective gradients, because they dedicate a large part of the evaluation to estimating the measure gradients. This difference suggests a possible extension to CMA-MEGA (TD3, ES) in which solutions are optimized with respect to the TD3 critic before being evaluated in the environment.

6.2.2 PGA-MAP-Elites and optimization efficiency. While there was no significant difference in the *final* QD scores of CMA-MEGA (TD3, ES) and PGA-MAP-Elites, CMA-MEGA (TD3, ES) was less *efficient* than PGA-MAP-Elites in some environments. For instance, in QD Hopper, PGA-MAP-Elites reached 1.5M QD score after 100k evaluations, but CMA-MEGA (TD3, ES) required 400k evaluations.

We can quantify optimization efficiency with *QD score AUC*, the area under the curve (AUC) of the QD score plot. For a QD algorithm which executes N iterations and evaluates λ solutions per iteration, we define QD score AUC as a Riemann sum:

$$\text{QD score AUC} = \sum_{i=1}^{N} (\lambda * \text{QD score at iteration } i) \tag{4}$$

After computing QD score AUC, we ran statistical analysis similar to Sec. 6.1 and found CMA-MEGA (TD3, ES) had significantly lower QD score AUC than PGA-MAP-Elites in QD Ant and QD Hopper. There was no significant difference in QD Half-Cheetah and QD Walker. As such, while CMA-MEGA (TD3, ES) obtained comparable final QD scores to PGA-MAP-Elites in all tasks, it was less efficient at achieving those scores in QD Ant and QD Hopper.

6.2.3 ME-ES and archive insertions. With one exception (CMA-MEGA (ES) in QD Hopper), both CMA-MEGA variants achieved significantly higher QD score than ME-ES in all environments. We attribute this result to the number of solutions each algorithm inserts into the archive. Each iteration, ME-ES evaluates 200 solutions (Appendix B) but only inserts one into the archive, for a total of 5000 solutions inserted during each run. Given that each archive has at least 1000 cells, ME-ES has, on average, 5 opportunities to insert a solution that improves each cell. In contrast, the CMA-MEGA variants have 100 times more insertions. Though the CMA-MEGA variants evaluate 200 solutions per iteration, they insert 100 of these into the archive. This totals to 500k insertions per run, allowing the CMA-MEGA variants to gradually improve archive cells.

6.2.4 MAP-Elites and robustness. In most cases, both CMA-MEGA variants had significantly higher QD score than MAP-Elites or no significant difference, but in QD Hopper, MAP-Elites achieved significantly higher QD score than CMA-MEGA (ES). However, we found that MAP-Elites solutions were less robust (see Appendix D).

6.2.5 CMA-MEGA variants and gradient estimates. In QD Hopper and QD Walker, CMA-MEGA (TD3, ES) had significantly higher QD score than CMA-MEGA (ES). One potential explanation is that PyBullet Gym (and hence QDGym) augments rewards with reward shaping signals intended to promote optimal solutions for deep RL algorithms. In prior work [45], these signals led PPO [54] to train successful walking agents, while they led OpenAI-ES into local optima. For instance, OpenAI-ES trained agents which stood still so as to maximize only the reward signal for staying upright.

Due to these signals, TD3's objective gradient seems more useful than that of OpenAI-ES in QD Hopper and QD Walker. In fact, the algorithms which performed best in QD Hopper and QD Walker were ones that calculated objective gradients with TD3, i.e. PGA-MAP-Elites and CMA-MEGA (TD3, ES).

Prior work [45] found that rewards could be tailored for ES, such that OpenAI-ES outperformed PPO. Extensions of our work could investigate whether there is a similar effect for QD algorithms, where tailoring the reward leads CMA-MEGA (ES) to outperform PGA-MAP-Elites and CMA-MEGA (TD3, ES).

7 CONCLUSION

To extend DQD to RL settings, we adapted gradient approximations from actor-critic methods and ES. By integrating these approximations with CMA-MEGA, we proposed two novel variants that we evaluated on four locomotion tasks from QDGym. CMA-MEGA (TD3, ES) performed comparably to the state-of-the-art PGA-MAP-Elites in all tasks but was less efficient in two of the tasks. CMA-MEGA (ES) performed comparably in two tasks.

Our results contrast prior work [19] where CMA-MEGA outperformed a baseline algorithm inspired by PGA-MAP-Elites in QD benchmark domains. The difference seems to be that difficulty in the benchmarks arises from a hard-to-explore measure space, whereas difficulty in QDGym arises from an objective which requires rigorous optimization. As such, future work could formalize the notions of "exploration difficulty" of a measure space and "optimization difficulty" of an objective and evaluate algorithms in benchmarks that cover a spectrum of these metrics.

For practitioners looking to apply DQD in RL settings, we recommend estimating objective gradients with an off-policy actor-critic method such as TD3 instead of with an ES. Due to the difficulty of modern control benchmarks, it is important to efficiently optimize the objective — TD3 benefits over ES since it can compute the objective gradient without further environment interaction. Furthermore, reward signals in these benchmarks are designed for deep RL methods, making TD3 gradients more useful than ES gradients.

By reducing QD-RL to DQD, we have decoupled QD-RL into DQD optimization and RL gradient approximations. In the future, we envision algorithms which benefit from advances in either more efficient DQD or more accurate RL gradient approximations.

ACKNOWLEDGMENTS

The authors thank the anonymous reviewers, Ya-Chuan Hsu, Heramb Nemlekar, and Gautam Salhotra for their invaluable feedback. This work was partially supported by the NSF NRI (#1053128) and NSF GRFP (#DGE-1842487).

REFERENCES

[1] Youhei Akimoto, Yuichi Nagata, Isao Ono, and Shigenobu Kobayashi. 2010. Bidirectional Relation between CMA Evolution Strategies and Natural Evolution Strategies. In *Parallel Problem Solving from Nature, PPSN XI*, Robert Schaefer, Carlos Cotta, Joanna Kołodziej, and Günter Rudolph (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 154–163.

[2] Shun-ichi Amari. 1998. Natural Gradient Works Efficiently in Learning. *Neural Computation* 10, 2 (02 1998), 251–276. https://doi.org/10.1162/089976698300017746 arXiv:https://direct.mit.edu/neco/article-pdf/10/2/251/813415/089976698300017746.pdf

[3] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, OpenAI Pieter Abbeel, and Wojciech Zaremba. 2017. Hindsight Experience Replay. In *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper/2017/file/453fadbd8a1a3af50a9df4df899537b5-Paper.pdf

[4] Hans-Georg Beyer and Hans-Paul Schwefel. 2002. Evolution strategies – A comprehensive introduction. *Natural Computing* 1, 1 (01 Mar 2002), 3–52. https://doi.org/10.1023/A:1015059928466

[5] Dimo Brockhoff, Anne Auger, Nikolaus Hansen, Dirk V. Arnold, and Tim Hohm. 2010. Mirrored Sampling and Sequential Selection for Evolution Strategies. In *Parallel Problem Solving from Nature, PPSN XI*, Robert Schaefer, Carlos Cotta, Joanna Kołodziej, and Günter Rudolph (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 11–21.

[6] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. 2016. OpenAI Gym. *CoRR* abs/1606.01540 (2016). arXiv:1606.01540 http://arxiv.org/abs/1606.01540

[7] Geoffrey Cideron, Thomas Pierrot, Nicolas Perrin, Karim Beguir, and Olivier Sigaud. 2020. QD-RL: Efficient Mixing of Quality and Diversity in Reinforcement Learning. *CoRR* abs/2006.08505 (2020). arXiv:2006.08505 https://arxiv.org/abs/2006.08505

[8] Jack Clark and Dario Amodei. 2016. Faulty Reward Functions in the Wild. https://openai.com/blog/faulty-reward-functions/.

[9] Cédric Colas, Vashisht Madhavan, Joost Huizinga, and Jeff Clune. 2020. Scaling MAP-Elites to Deep Neuroevolution. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference* (Cancún, Mexico) *(GECCO '20)*. Association for Computing Machinery, New York, NY, USA, 67–75. https://doi.org/10.1145/3377930.3390217

[10] Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. 2018. GEP-PG: Decoupling Exploration and Exploitation in Deep Reinforcement Learning Algorithms. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*, Jennifer Dy and Andreas Krause (Eds.). PMLR, 1039–1048. https://proceedings.mlr.press/v80/colas18a.html

[11] Edoardo Conti, Vashisht Madhavan, Felipe Petroski Such, Joel Lehman, Kenneth Stanley, and Jeff Clune. 2018. Improving Exploration in Evolution Strategies for Deep Reinforcement Learning via a Population of Novelty-Seeking Agents. In *Advances in Neural Information Processing Systems 31*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.). Curran Associates, Inc., 5027–5038. http://papers.nips.cc/paper/7750-improving-exploration-in-evolution-strategies-for-deep-reinforcement-learning-via-a-population-of-novelty-seeking-agents.pdf

[12] Erwin Coumans and Yunfei Bai. 2016–2020. PyBullet, a Python module for physics simulation for games, robotics and machine learning. http://pybullet.org.

[13] Antoine Cully, Jeff Clune, Danesh Tarapore, and Jean-Baptiste Mouret. 2015. Robots that can adapt like animals. *Nature* 521 (05 2015), 503–507. https://doi.org/10.1038/nature14422

[14] Pieter-Tjerk de Boer, Dirk P. Kroese, Shie Mannor, and Reuven Y. Rubinstein. 2005. A Tutorial on the Cross-Entropy Method. *Annals of Operations Research* 134, 1 (01 Feb 2005), 19–67. https://doi.org/10.1007/s10479-005-5724-z

[15] Benjamin Ellenberger. 2018–2019. PyBullet Gymperium. https://github.com/benelot/pybullet-gym.

[16] Benjamin Eysenbach, Abhishek Gupta, Julian Ibarz, and Sergey Levine. 2019. Diversity is All You Need: Learning Skills without a Reward Function. In *International Conference on Learning Representations*. https://openreview.net/forum?id=SJx63jRqFm

[17] Matthew Fontaine and Stefanos Nikolaidis. 2021. A Quality Diversity Approach to Automatically Generating Human-Robot Interaction Scenarios in Shared Autonomy. *Robotics: Science and Systems* (2021).

[18] Matthew C. Fontaine, Ya-Chuan Hsu, Yulun Zhang, Bryon Tjanaka, and Stefanos Nikolaidis. 2021. On the Importance of Environments in Human-Robot Coordination. *Robotics: Science and Systems* (2021).

[19] Matthew C. Fontaine and Stefanos Nikolaidis. 2021. Differentiable Quality Diversity. *Advances in Neural Information Processing Systems* 34 (2021). https://proceedings.neurips.cc/paper/2021/file/532923f11ac97d3e7cb0130315b067dc-Paper.pdf

[20] Matthew C. Fontaine, Julian Togelius, Stefanos Nikolaidis, and Amy K. Hoover. 2020. Covariance Matrix Adaptation for the Rapid Illumination of Behavior Space. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference* (Cancún, Mexico) *(GECCO '20)*. Association for Computing Machinery, New York, NY, USA, 94–102. https://doi.org/10.1145/3377930.3390232

[21] Scott Fujimoto, Herke van Hoof, and David Meger. 2018. Addressing Function Approximation Error in Actor-Critic Methods. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*, Jennifer Dy and Andreas Krause (Eds.). PMLR, 1587–1596. http://proceedings.mlr.press/v80/fujimoto18a.html

[22] Adam Gaier, Alexander Asteroth, and Jean-Baptiste Mouret. 2018. Data-efficient design exploration through surrogate-assisted illumination. *Evolutionary computation* 26, 3 (2018), 381–410.

[23] Adam Gaier, Alexander Asteroth, and Jean-Baptiste Mouret. 2020. Discovering Representations for Black-Box Optimization. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference* (Cancún, Mexico) *(GECCO '20)*. Association for Computing Machinery, New York, NY, USA, 103–111. https://doi.org/10.1145/3377930.3390221

[24] Xavier Glorot and Yoshua Bengio. 2010. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 9)*, Yee Whye Teh and Mike Titterington (Eds.). PMLR, Chia Laguna Resort, Sardinia, Italy, 249–256. https://proceedings.mlr.press/v9/glorot10a.html

[25] Daniele Gravina, Ahmed Khalifa, Antonios Liapis, Julian Togelius, and Georgios N Yannakakis. 2019. Procedural content generation through quality diversity. In *2019 IEEE Conference on Games (CoG)*. IEEE, 1–8.

[26] David Ha. 2017. A Visual Guide to Evolution Strategies. *blog.otoro.net* (2017). https://blog.otoro.net/2017/10/29/visual-evolution-strategies/

[27] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *Proceedings of the 35th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 80)*, Jennifer Dy and Andreas Krause (Eds.). PMLR, 1861–1870. https://proceedings.mlr.press/v80/haarnoja18b.html

[28] Nikolaus Hansen. 2016. The CMA Evolution Strategy: A Tutorial. *CoRR* abs/1604.00772 (2016). arXiv:1604.00772 http://arxiv.org/abs/1604.00772

[29] Alex Irpan. 2018. Deep Reinforcement Learning Doesn't Work Yet. https://www.alexirpan.com/2018/02/14/rl-hard.html.

[30] Shauharda Khadka, Somdeb Majumdar, Tarek Nassar, Zach Dwiel, Evren Tumer, Santiago Miret, Yinyin Liu, and Kagan Tumer. 2019. Collaborative Evolutionary Reinforcement Learning. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 3341–3350. https://proceedings.mlr.press/v97/khadka19a.html

[31] Shauharda Khadka and Kagan Tumer. 2018. Evolution-Guided Policy Gradient in Reinforcement Learning. In *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.), Vol. 31. Curran Associates, Inc. https://proceedings.neurips.cc/paper/2018/file/85fc37b18c57097425b52fc7afbb6969-Paper.pdf

[32] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). http://arxiv.org/abs/1412.6980

[33] Saurabh Kumar, Aviral Kumar, Sergey Levine, and Chelsea Finn. 2020. One Solution is Not All You Need: Few-Shot Extrapolation via Structured MaxEnt RL. *Advances in Neural Information Processing Systems* 33 (2020).

[34] Joel Lehman, Jay Chen, Jeff Clune, and Kenneth O. Stanley. 2018. ES is More than Just a Traditional Finite-Difference Approximator. In *Proceedings of the Genetic and Evolutionary Computation Conference* (Kyoto, Japan) *(GECCO '18)*. Association for Computing Machinery, New York, NY, USA, 450–457. https://doi.org/10.1145/3205455.3205474

[35] Joel Lehman and Kenneth O. Stanley. 2011. Abandoning Objectives: Evolution Through the Search for Novelty Alone. *Evolutionary Computation* 19, 2 (06 2011), 189–223. https://doi.org/10.1162/EVCO_a_00025 arXiv:https://direct.mit.edu/evco/article-pdf/19/2/189/1494066/evco_a_00025.pdf

[36] Joel Lehman and Kenneth O. Stanley. 2011. Evolving a Diversity of Virtual Creatures through Novelty Search and Local Competition. In *Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation* (Dublin, Ireland) *(GECCO '11)*. Association for Computing Machinery, New York, NY, USA, 211–218. https://doi.org/10.1145/2001576.2001606

[37] Yunzhu Li, Jiaming Song, and Stefano Ermon. 2017. InfoGAIL: Interpretable Imitation Learning from Visual Demonstrations. In *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. https://proceedings.neurips.cc/paper/2017/file/2cd4e8a2ce081c3d7c32c3cde4312ef7-Paper.pdf

[38] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2016. Continuous control with deep reinforcement learning. In *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Yoshua Bengio and Yann LeCun (Eds.). http://arxiv.org/abs/1509.02971

[39] Horia Mania, Aurelia Guy, and Benjamin Recht. 2018. Simple Random Search of Static Linear Policies is Competitive for Reinforcement Learning. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (Montréal, Canada) *(NIPS'18)*. Curran Associates Inc., Red Hook, NY, USA, 1805–1814.

[40] Jean-Baptiste Mouret and Jeff Clune. 2015. Illuminating search spaces by mapping elites. *CoRR* abs/1504.04909 (2015). arXiv:1504.04909 http://arxiv.org/abs/1504.04909

[41] Andrew Y. Ng, Daishi Harada, and Stuart J. Russell. 1999. Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML '99)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 278–287.

[42] Olle Nilsson. 2021. QDgym. https://github.com/ollenilsson19/QDgym.

[43] Olle Nilsson and Antoine Cully. 2021. Policy Gradient Assisted MAP-Elites. In *Proceedings of the Genetic and Evolutionary Computation Conference* (Lille, France) *(GECCO '21)*. Association for Computing Machinery, New York, NY, USA, 866–875. https://doi.org/10.1145/3449639.3459304

[44] OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. 2019. Solving Rubik's Cube with a Robot Hand. *arXiv preprint* (2019).

[45] Paolo Pagliuca, Nicola Milano, and Stefano Nolfi. 2020. Efficacy of Modern Neuro-Evolutionary Strategies for Continuous Control Optimization. *Frontiers in Robotics and AI* 7 (2020), 98. https://doi.org/10.3389/frobt.2020.00098

[46] Jack Parker-Holder, Aldo Pacchiano, Krzysztof M Choromanski, and Stephen J Roberts. 2020. Effective Diversity in Population Based Reinforcement Learning. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 18050–18062. https://proceedings.neurips.cc/paper/2020/file/d1dc3a8270a6f9394f88847d7f0050cf-Paper.pdf

[47] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. 2018. Sim-to-Real Transfer of Robotic Control with Dynamics Randomization. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*. 3803–3810. https://doi.org/10.1109/ICRA.2018.8460528

[48] Pourchot and Sigaud. 2019. CEM-RL: Combining evolutionary and gradient-based methods for policy search. In *International Conference on Learning Representations*. https://openreview.net/forum?id=BkeU5j0ctQ

[49] Justin K. Pugh, Lisa B. Soros, and Kenneth O. Stanley. 2016. Quality Diversity: A New Frontier for Evolutionary Computation. *Frontiers in Robotics and AI* 3 (2016), 40. https://doi.org/10.3389/frobt.2016.00040

[50] Nemanja Rakicevic, Antoine Cully, and Petar Kormushev. 2021. Policy Manifold Search: Exploring the Manifold Hypothesis for Diversity-Based Neuroevolution. In *Proceedings of the Genetic and Evolutionary Computation Conference* (Lille, France) *(GECCO '21)*. Association for Computing Machinery, New York, NY, USA, 901–909. https://doi.org/10.1145/3449639.3459320

[51] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. 2017. Evolution Strategies as a Scalable Alternative to Reinforcement Learning. arXiv:1703.03864 [stat.ML]

[52] Tom Schaul, Daniel Horgan, Karol Gregor, and David Silver. 2015. Universal Value Function Approximators. In *Proceedings of the 32nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 37)*, Francis Bach and David Blei (Eds.). PMLR, Lille, France, 1312–1320. https://proceedings.mlr.press/v37/schaul15.html

[53] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. 2015. Trust Region Policy Optimization. In *Proceedings of the 32nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 37)*, Francis Bach and David Blei (Eds.). PMLR, Lille, France, 1889–1897. https://proceedings.mlr.press/v37/schulman15.html

[54] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *CoRR* abs/1707.06347 (2017). arXiv:1707.06347 http://arxiv.org/abs/1707.06347

[55] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (second ed.). The MIT Press. http://incompleteideas.net/book/the-book-2nd.html

[56] Yunhao Tang. 2021. Guiding Evolutionary Strategies with Off-Policy Actor-Critic. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems* (Virtual Event, United Kingdom) *(AAMAS '21)*. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 1317–1325.

[57] Bryon Tjanaka, Matthew C. Fontaine, Yulun Zhang, Sam Sommerer, Nathan Dennler, and Stefanos Nikolaidis. 2021. pyribs: A bare-bones Python library for quality diversity optimization. https://github.com/icaros-usc/pyribs.

[58] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. 2017. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 23–30. https://doi.org/10.1109/IROS.2017.8202133

[59] Vassilis Vassiliades and Jean-Baptiste Mouret. 2018. Discovering the Elite Hypervolume by Leveraging Interspecies Correlation. In *Proceedings of the Genetic and Evolutionary Computation Conference* (Kyoto, Japan) *(GECCO '18)*. Association for Computing Machinery, New York, NY, USA, 149–156. https://doi.org/10.1145/3205455.3205602

[60] Daan Wierstra, Tom Schaul, Tobias Glasmachers, Yi Sun, Jan Peters, and Jürgen Schmidhuber. 2014. Natural Evolution Strategies. *Journal of Machine Learning Research* 15, 27 (2014), 949–980. http://jmlr.org/papers/v15/wierstra14a.html

[61] Daan Wierstra, Tom Schaul, Jan Peters, and Juergen Schmidhuber. 2008. Natural Evolution Strategies. In *2008 IEEE Congress on Evolutionary Computation (IEEE World Congress on Computational Intelligence)*. 3381–3387. https://doi.org/10.1109/CEC.2008.4631255